

Session 5: Advanced Lists, Comprehensions & Data Structures

Take-Home Practice Problems

Introduction

Welcome to Session 05 Practice!

These problems will help you master the key concepts from Session 05:

Topics Covered:

- Advanced list operations: slicing, methods, copying
- List comprehensions for elegant transformations
- Dictionaries for key-value storage
- Tuples for immutable data
- Sets for unique collections and set operations

What Makes These Problems Better:

- ✓ **Clear requirements** - Each function has specific inputs and outputs
- ✓ **Built-in tests** - Run your code and see if it passes!
- ✓ **Formatted output** - All printing is handled for you
- ✓ **Focus on logic** - Just write the function bodies

How to Complete the Problems

1. Open the `take_home_questions` folder in the `session05` directory
2. You'll find 4 Python files, one for each question
3. Each file has:
 - Function templates with clear documentation
 - TODO comments showing where to write your code
 - Test code that runs automatically
 - Expected output examples
4. Complete each function by replacing `pass` with your code
5. Run the file to test your implementation

6. Don't modify the test code at the bottom!

Question 1: List Slicer

File: `question1_list_slicer.py`

Topics: List slicing, list methods, list copying

What You'll Do:

Implement four functions to manipulate lists:

1. `get_first_three()` - Use slicing to get first 3 items
2. `get_every_second()` - Use slicing with step to get every other item
3. `reverse_list()` - Use negative step to reverse a list
4. `safe_copy_and_modify()` - Create a copy and modify it safely

Key Concepts:

- Slicing syntax: `list[start:stop:step]`
- Reversing with `[::-1]`
- **CRITICAL:** Creating copies with `.copy()` or `[:]`
- Understanding references vs copies

Example Output:

```
=====
SESSION 05 - QUESTION 1: LIST SLICER
=====
```

Original list: [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

TEST 4: Copy and modify (original should NOT change!)

Original before: [1, 2, 3, 4, 5]

Modified copy: [1, 2, 3, 4, 5, 999]

Original after: [1, 2, 3, 4, 5]

Modified copy correct? PASS

Original unchanged? PASS

Question 2: Comprehension Builder

File: question2_comprehension_builder.py

Topics: List comprehensions with filtering

What You'll Do:

Implement four functions using **list comprehensions**:

1. `square_all_numbers()` - Square all numbers in one line
2. `filter_positive()` - Keep only positive numbers
3. `double_evens()` - Filter AND transform even numbers
4. `extract_names()` - Extract names from dictionaries

Key Concepts:

- Basic comprehension: `[expression for item in list]`
- With condition: `[expression for item in list if condition]`
- Comprehensions are more Pythonic than map/filter
- Can access dictionary keys: `[d["key"] for d in list]`

Example Output:

```
=====
SESSION 05 - QUESTION 2: COMPREHENSION BUILDER
=====

TEST 3: Double only the even numbers
-----
Input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Result: [4, 8, 12, 16, 20]
Expected: [4, 8, 12, 16, 20]
Status: PASS
```

Question 3: Contact Book

File: question3_contact_book.py

Topics: Dictionaries (creating, accessing, modifying, looping)

What You'll Do:

Implement five functions to build a contact book:

1. `create_contact()` - Create a contact dictionary
2. `get_contact_info()` - Safely access data with `.get()`

3. `update_contact()` - Modify dictionary values
4. `count_contacts()` - Return summary dictionary
5. `find_all_names()` - Extract all names from contacts

Key Concepts:

- Creating dictionaries: `{"key": value}`
- Safe access with `.get(key, default)`
- Modifying: `dict[key] = new_value`
- Extracting values from list of dicts

Example Output:

```
=====
SESSION 05 - QUESTION 3: CONTACT BOOK
=====

TEST 3: Get missing contact information
-----
Address: Not found
Status: PASS
```

Question 4: Data Structures (Tuples & Sets)

File: `question4_data_structures.py`

Topics: Tuples (immutable), Sets (unique items, operations)

What You'll Do:

Implement five functions working with tuples and sets:

1. `create_coordinate()` - Create a coordinate tuple
2. `unpack_person()` - Unpack tuple values
3. `remove_duplicates()` - Use set to remove duplicates
4. `find_common_items()` - Use set intersection
5. `find_unique_items()` - Use set difference

Key Concepts:

- Tuples are immutable: `(x, y)`
- Tuple unpacking: `name, age = person_tuple`
- Sets remove duplicates: `set([1, 1, 2])` becomes `{1, 2}`
- Set operations: `&` (intersection), `-` (difference)

Example Output:

```
=====
SESSION 05 - QUESTION 4: DATA STRUCTURES
=====

TEST 4: Find common items (intersection)
-----
Alice's courses: ['Math', 'Physics', 'Chemistry', 'Biology']
Bob's courses: ['Physics', 'Chemistry', 'History', 'Art']
Common courses: {'Physics', 'Chemistry'}
Status: PASS
```

Success Criteria

✓ You've Mastered Session 05 If:

- All tests pass with PASS status
- You can slice lists confidently (`[:3]`, `[:2]`, `[:-1]`)
- You understand list copying vs references
- You can write list comprehensions fluently
- You know when to use dictionaries vs lists
- You can safely access dictionary values with `.get()`
- You understand tuple immutability
- You can use set operations for finding common/unique items

Common Mistakes to Avoid

Watch Out!

1. Not copying lists before modifying

```
1 # Wrong - both variables point to same list!
2 list2 = list1
3 list2.append(4) # list1 also changes!
4
5 # Right - create a copy
6 list2 = list1.copy() # or list1[:]
7 list2.append(4) # list1 unchanged
8
```

2. Using [] instead of .get() for dictionaries

```
1 # Risky - raises KeyError if key missing
2 value = contact["email"] # Error if no "email" key!
3
4 # Safe - returns default if key missing
5 value = contact.get("email", "Not found")
6
```

3. Forgetting about immutable tuples

```
1 coord = (10, 20)
2 # coord[0] = 15 # TypeError! Can't modify tuples
3
```

4. Creating empty set incorrectly

```
1 # Wrong - this creates an empty dict!
2 my_set = {}
3
4 # Right
5 my_set = set()
6
```

5. Forgetting to convert set back to list

```
1 # If function should return list, not set:
2 unique = list(set(items)) # Convert back to list
3
```

Key Concepts Reference

Quick Reference

List Slicing:

```
1 nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2
3 nums[:3]      # [0, 1, 2] - first 3
4 nums[-3:]     # [7, 8, 9] - last 3
5 nums[::2]     # [0, 2, 4, 6, 8] - every 2nd
6 nums[::-1]    # [9, 8, 7, ..., 0] - reversed
```

List Comprehensions:

```
1 # Basic
2 squared = [x**2 for x in numbers]
3
4 # With condition
5 evens = [x for x in numbers if x % 2 == 0]
6
7 # Transform and filter
8 doubled_evens = [x*2 for x in numbers if x % 2 == 0]
```

Dictionary Operations:

```
1 # Create
2 person = {"name": "Alice", "age": 25}
3
4 # Access (safe)
5 name = person.get("name", "Unknown")
6
7 # Update
8 person["age"] = 26
9
10 # Loop
11 for key, value in person.items():
12     print(f"{key}: {value}")
```

Tuples:

```
1 # Create
2 coord = (10, 20)
3
4 # Unpack
5 x, y = coord
6
7 # Immutable - can't change!
```

Sets:

```
1 set1 = {1, 2, 3, 4}
2 set2 = {3, 4, 5, 6}
3
4 set1 & set2  # {3, 4} - intersection (common)
5 set1 | set2  # {1,2,3,4,5,6} - union (all)
6 set1 - set2  # {1, 2} - difference (set1 only)
7
8 # Remove duplicates
9 unique = list(set([1, 2, 2, 3, 3])) # [1, 2, 3]
```

Testing Your Code

To run each question:

```
cd session05/take_home_questions
python question1_list_slicer.py
python question2_comprehension_builder.py
python question3_contact_book.py
python question4_data_structures.py
```

Each file will:

- Run multiple tests automatically
- Show expected vs actual output
- Display PASS or FAIL for each test
- Include bonus examples to explore

Choosing the Right Data Structure

When to Use Each Structure

LIST - [1, 2, 3]

- Ordered collection that can change
- Allows duplicates
- Use when: You need to maintain order and may add/remove items
- Example: Shopping list, scores, history

TUPLE - (1, 2, 3)

- Ordered collection that **cannot** change
- Allows duplicates
- Use when: Data should never change
- Example: Coordinates (x, y), RGB colors, database records

SET - {1, 2, 3}

- Unordered collection
- **No duplicates** - automatically removed
- Use when: You need unique items or set operations
- Example: Tags, unique visitors, common courses

DICTIONARY - {"name": "Alice", "age": 25}

- Key-value pairs
- Fast lookups by key ($O(1)$)
- Use when: You need to look up data by name/id
- Example: Contacts, settings, counters

Next Steps After Completion

1. **Experiment:** Try modifying the test data
2. **Challenge yourself:** Look at the bonus sections
3. **Review:** If tests fail, review the function documentation
4. **Practice:** Try writing your own test cases
5. **Combine:** Try using multiple data structures together

Final Encouragement

Remember:

- **Read carefully:** Each function has clear documentation
- **Test incrementally:** Complete and test one function at a time
- **Use the right tool:** Choose the appropriate data structure
- **Copy carefully:** Remember that `list2 = list1` doesn't create a copy!
- **Ask questions:** If you're stuck, reach out!

These problems build on Session 04 and introduce powerful new tools for organizing data. Master these concepts and you'll be able to solve much more complex problems! 😊

Questions? Need help? Don't hesitate to reach out!